Restaurant Management System - Backend Implementation

Advanced Database Management System

By Mahreen, Samia and Unzila
(BSCS24049 , BSCS24121, BSCS24154)

1. System Architecture

The backend uses an **MVC-style structure adapted for a REST API**. Since it's an API, there is no traditional **View layer**—the server simply returns **JSON responses**.

When a client sends a request, it goes through several steps. First, the request reaches **server.js**, where global middleware like **helmet**, **cors**, and the JSON parser run. Then the request URL is matched with routes defined in the **routes** folder. After that, the **authentication middleware** checks the JWT token for protected routes, and **role-based middleware** verifies user permissions. The request body is also validated using **Joi schemas**. If everything is valid, the **controller** runs the main business logic and interacts with the database. Finally, the server sends a **JSON response**, or an error is handled by a centralized error handler.

The project is organized in a clear structure. **server.js** acts as the entry point and registers middleware and routes. **package.json** lists dependencies, and **.env.example** shows required environment variables. Inside the **src folder**, there are configs for database and JWT settings, controllers for handling requests, middleware for authentication and access control, routes for API endpoints, schemas for validation, and utils for helpers like logging. The **logs folder** stores transaction rollback logs.

Overall, this structure keeps everything organized: **routes manage endpoints, middleware handles shared tasks, and controllers contain the main logic.**

---

2. Authentication Flow

Authentication uses **stateless JWT tokens** instead of sessions. This means the server doesn't store login sessions, which makes the system simpler and easier to scale.

When a user logs in through **POST /api/v1/auth/login**, the server receives the email (or username) and password. It searches the database for the user using a secure parameterized query. Then **bcrypt.compare** checks the entered password against the stored hashed password. If the credentials are correct, **JWT.sign** creates a token. This token contains details like the user ID, username, user type (employee or admin), role, role_id, issue time, expiration time, and an issuer value (restaurant-api). The token is then sent back to the client.

Protected routes use an **authenticate middleware**. This middleware reads the token from the **Authorization Bearer header**, verifies it using the secret key, checks that it hasn't expired, and confirms the issuer. If everything is valid, the decoded user data is attached to req.user. Otherwise, the request is rejected with a **401 error**.

After authentication, **role-based middleware (requireRole)** checks permissions before the controller runs. If req.user is missing, it returns **401**. If the user is an admin, it checks whether admin access is allowed. For employees, it compares the user's role with the allowed roles for that route. If the role doesn't match, a **403 error** is returned.

Different API endpoints require different roles:

- ➢ **POST /orders** → Waiter
- ➢ **POST /orders/id/process-inventory** → Chef
- ➢ **POST /payments** → Cashier
- ➢ **PUT /inventory** → Chef
- ➢ **POST /shifts** → Admin
- ➢ **GET /reports** → Admin
- ➢ **POST /auth/register** → Super Admin

Some endpoints like **GET orders, menu, and customers** can be accessed by any authenticated user, while **GET /health** is public and requires no token.

Overall, all roles from Phase 1 — **Chef, Waiter, Cashier, and Admin (Super and Normal)** — are enforced through middleware to ensure proper access control.

---

3. Transaction Management

ACID compliance is important for restaurant operations because partial updates can cause serious problems. For example, an order might be saved but inventory may not be deducted, or the same table could be booked twice. To avoid this, all transaction scenarios use explicit **BEGIN, COMMIT, and ROLLBACK** with dedicated database connections.

The project also requires using **raw SQL instead of relying only on an ORM**. Raw SQL was chosen because it gives full control over transactions. Using **BEGIN, COMMIT, and ROLLBACK on the same connection** removes ambiguity and ensures consistency. It also allows specifying isolation levels like **SERIALIZABLE** or using table hints such as **UPDLOCK**. This approach clearly shows how database transactions actually work.

Database connections are managed using the **mssql library connection pool** configured in `database.js`. The pool includes settings like user, password, server, and database name. It allows a maximum of **10 connections**, keeps **2 minimum connections active**, and closes idle connections after **30 seconds**.

There are two ways the pool is used. For **simple queries**, `getPool.request()` automatically takes a free connection, runs the query, and returns the connection to the pool. For **transactions**, `getTransaction()` creates a new `sql.Transaction`, which reserves a dedicated connection for the entire transaction until it finishes with **COMMIT or ROLLBACK**.

All transaction controllers follow a similar pattern. A transaction starts inside a **try block**. A request is created from the transaction, then the system verifies data, checks business rules, performs inserts or updates, and commits if everything succeeds. If an error occurs, the **catch block** runs `transaction.rollback()`, logs the rollback with details, and returns the

correct HTTP error. **404** is used for not found errors, **409** for conflicts, and other errors go to the centralized error handler.

Every rollback is recorded in **`logs/transactions.log`** with a timestamp, scenario name, error details, and related IDs. This creates a clear audit trail for debugging and verification.

Transaction Scenario: Create Order

The **Create Order transaction** makes sure an order and all its items are saved **together as one unit**. If any dish in the order does not exist, the whole operation is cancelled and nothing is saved.

First, the transaction checks if the **customer exists** in the Customer table using the given customer_id. Then it checks if the **employee exists** in the Employee table. If both exist, the system inserts the **order header** into the Order table with the current time (SYSDATETIME), customer_id, and employee_id. The database returns the new **order_id**, which will be used for the order items.

After that, each dish in the order is processed. The system checks if the **dish exists** in MenuDish. If a dish is missing, an error is thrown and the transaction stops. If it exists, the item is inserted into the OrderItem table with order_id, dish_id, and quantity. When all items are inserted successfully, the transaction **commits**.

If something goes wrong, the transaction **rolls back**. For example:

- Customer not found → **404**
- Employee not found → **404**
- Dish not found → **404** with the dish name
- Any database error → **500**

The reason for using a transaction is to avoid **partial data**. For example, if an order had five dishes and the third one was invalid, the order header and first two items could still be saved without a transaction. This would create an **incomplete order and messy data**. With a transaction, either **everything saves successfully or nothing saves at all**, keeping the database consistent.

Transaction Scenario: Table Reservation

The **Table Reservation transaction** prevents **double booking** when multiple customers try to reserve the same table at the same time.

Without proper locking, two requests could check the table at the same moment, both see it as **Available**, and both create reservations. This would result in **overbooking**.

To prevent this, the transaction uses the **UPDLOCK hint**. It starts by selecting the table from DiningTable using **UPDLOCK and ROWLOCK** where table_id matches. This locks that specific row. The system then checks the table status. If the status is not **Available**, it throws an error.

If the table is available, the system inserts a record into the Reservation table with customer_id, table_id, and reservation_time, and captures the new reservation_id. Then it updates the DiningTable status to **Reserved** and commits the transaction.

Here is what happens with two requests:

- **Request A** selects the table with UPDLOCK, which locks that row.
- **Request B** tries to run the same query but must wait because the row is locked.
- Request A finishes the transaction and releases the lock.
- Request B then continues, sees the table is now **Reserved**, and returns **409 Conflict**.

This method was chosen because it is simple and reliable. Other options like **application-level locks (e.g., Redis)** would add extra complexity. **Optimistic locking** could still allow both requests to run and fail later. **Table-level locks** would block too many operations. Using **UPDLOCK** locks only the needed row and works safely within the transaction.

Transaction Scenario: Shift Assignment

The **Shift Assignment transaction** assigns a shift to an employee only if they **don't already have another shift at the same time**.

First, the transaction checks if the **employee exists** in the Employee table using the given employee_id. After that, it checks for **overlapping shifts**. It searches the Shift table for any record with the same employee_id where the new shift time overlaps with an existing shift. If any record is found, it means the employee already has a shift at that time, so an **overlapping shift error** is returned.

If there is no overlap, the system inserts the new shift into the Shift table with employee_id, start_time, and end_time. The database returns the new **shift_id**, and then the transaction **commits**.

Using a transaction is important here. Without it, two requests could check for overlaps at the same time and both see that no shift exists. Then both could insert shifts, causing the employee to be **double-booked**. The transaction makes sure the **check and insert happen together**, so no other request can insert a shift in between.

3.4 Transaction Scenario: Process Inventory

The **Process Inventory transaction** deducts ingredient stock for all dishes in a completed order. If even one ingredient does not have enough stock, **nothing is deducted**, and the system reports all shortages.

This transaction is more complex because an order can contain **many dishes and many ingredients**, and stock levels might change while the process is running. To prevent issues like new inventory rows appearing during the check, the transaction uses the **SERIALIZABLE isolation level**. This prevents **phantom reads** and keeps the data stable during the transaction.

First, the system gets all items from OrderItem for the given order_id. For each dish, it finds its ingredients by joining MenuDishIngredient with Ingredient. Then

for each ingredient, it selects the current stock from Inventory using **UPDLOCK**, which locks that row.

The system compares the **required quantity** with the **available quantity**. If any ingredient is short, the shortage is recorded but the process continues checking the rest so that **all shortages can be reported together**. After checking everything, if any shortages exist, an error is thrown and the transaction **rolls back**.

If all ingredients have enough stock, the system updates the Inventory table by subtracting the required amount for each ingredient, and then **commits** the transaction.

There is also a **CHECK constraint** on the Inventory table that ensures quantity can never go below zero. So even if a bug in the application tried to deduct too much, the **database itself would reject it**, adding an extra layer of protection.

Here is a **shorter, simpler, and more natural version** written in basic English like your style.

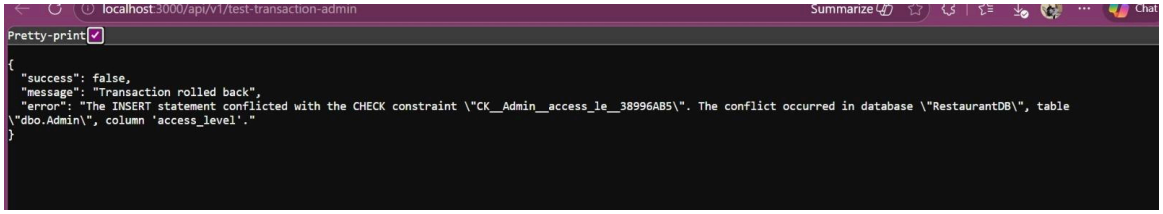
---

4. Transaction Rollback Demonstration with CHECK Constraint

To show how rollback works, I created a **test API endpoint** that tries to insert an admin with an invalid access_level.
In the database, there is a **CHECK constraint** that only allows two values: **'Super'** or **'Normal'**.

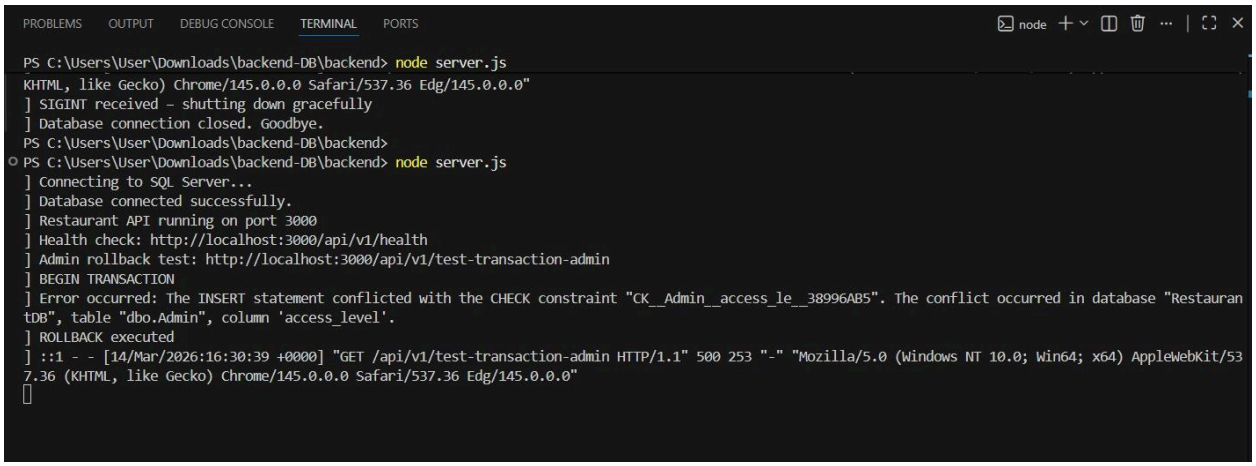
When the transaction starts, it tries to insert an admin with access_level = 'Invalid'. Because this value is not allowed, the **database rejects the insert**. The transaction then **rolls back**, and no record is saved.

Screenshot 1: API Response



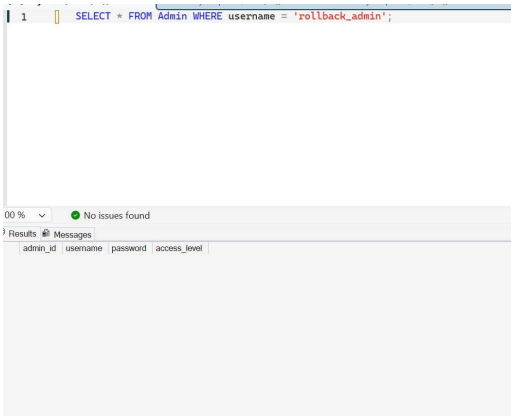
This screenshot shows the **API response** when the transaction fails.
The response shows success: false and a message that the **transaction was rolled back**.
The error also says that the INSERT conflicted with the **CHECK constraint in the Admin table** on the access_level column.
This means the client clearly knows **why the transaction failed**.

Screenshot 2: Server Console



This screenshot shows the **server console output** during the transaction.
The console first shows **BEGIN TRANSACTION**, then the error about the CHECK constraint, and finally **ROLLBACK executed**.
This confirms the rollback actually happened. The log also shows **HTTP 500**, which means the client received the error response.

Screenshot 3: Database Query



This screenshot shows a **database query after the failed transaction**.
When I run SELECT * FROM Admin WHERE username = rollback_admin, the query returns **no results**.
This proves the insert was **fully rolled back** and no data was saved.

This test proves that:

- ★ The transaction started with **BEGIN**
- ★ The CHECK constraint detected invalid data
- ★ The transaction **rolled back correctly**
- ★ The database stayed **clean with no partial data**
- ★ The client received a **clear error message**

The **CHECK constraint** also works as an extra safety layer. Even if the application code missed the error, the **database would still block invalid data**.

---

5. Security Practices

SQL Injection Protection

SQL injection is prevented by using **parameterized queries**.
User input is never added directly into SQL strings. The SQL driver safely handles parameters, so even inputs like "OR 1=1 --" are treated as normal text.

Password Security

Passwords are hashed using **bcrypt with 12 salt rounds**.
This makes brute-force attacks slow. Passwords are **never stored in plaintext**, never logged, and never returned in API responses.

JWT Security

JWT tokens are signed using a secret stored in the **JWT_SECRET environment variable**.
Tokens expire after **8 hours** and include only necessary data like **id, username, type, role, and role_id**.
The token also includes an **issuer value (restaurant-api)**, which is checked when verifying the token.

HTTP Security

Security headers are added using **helmet middleware**, including:

- ★ X-Content-Type-Options: nosniff
- ★ X-Frame-Options: DENY
- ★ Strict-Transport-Security
- ★ Content-Security-Policy

Input Validation

All requests are validated using **Joi schemas** before reaching the controllers.
This checks for missing fields, wrong data types, invalid values, and out-of-range numbers.
If validation fails, the API returns **400 Bad Request** with clear error messages.

Benefits:

- ★ Errors are caught early
- ★ Users get clear messages
- ★ Invalid SQL queries are avoided
- ★ Request structure is clearly defined

---

6. Key Design Decisions and Tradeoffs

Raw SQL vs ORM

Raw SQL was used instead of ORMs like Sequelize or TypeORM because the project requires **manual transaction control**.
Raw SQL also makes complex joins easier and allows using **SQL Server features** like UPDLOCK and SERIALIZABLE.

The downside is **more code and manual mapping**, but it gives **full control and better understanding** of database behavior.

Choosing the mssql Library

The **mssql library** was used because the project uses **SQL Server specific features**, such as:

- IDENTITY for auto-increment
- OUTPUT INSERTED for returning IDs
- NVARCHAR
- DATETIME2
- SYSDATETIME
- UPDLOCK

Other libraries like mysql2 or pg would not match these features.

Using UPDLOCK for Reservations

UPDLOCK was chosen to prevent double bookings.
Other options like Redis locks or optimistic locking would add complexity or waste work.
UPDLOCK locks only the needed row and automatically releases the lock after commit or rollback.

The tradeoff is **slightly lower concurrency**, but reservations are not very frequent.

SERIALIZABLE for Inventory

SERIALIZABLE isolation was used for inventory deduction to prevent **phantom reads**.
This ensures inventory checks and updates happen **safely and atomically**.

The tradeoff is more locking, but inventory operations are **less frequent**, so this is acceptable.

Server-Side Payment Calculation

The client **never sends the payment amount**.
Instead, the server calculates it using **quantity × price from the database**.
This prevents users from modifying the price.

Separate Admin and Employee Authentication

Admins and employees are treated as **different identity types**.
Admins use **access_level**, while employees use **roles**.

JWT tokens include a type field so the system knows if the user is **admin or employee**.
This prevents role confusion.

Centralized Error Handling

All controllers send errors to a **single errorHandler middleware**.
This ensures all APIs return **consistent JSON error responses**.
In development, errors include stack traces. In production, sensitive details are hidden.

---

7. Rollback Evidence Summary

The three screenshots together prove that rollback works correctly.

- **Screenshot 1** shows the **client response** with the rollback message and database error.
- **Screenshot 2** shows the **server console**, confirming the transaction started and rolled back.
- **Screenshot 3** shows the **database state**, proving no record was saved.

Together, they clearly demonstrate that the transaction system works properly.

---

8. Conclusion

This backend successfully meets all **Phase 2 requirements**.

It provides a **REST API with proper HTTP methods, status codes, JSON responses, and versioning** under /api/v1.
Authentication uses **JWT with bcrypt**, and authorization uses **RBAC middleware** for all roles from Phase 1.

Transactions are implemented with **BEGIN, COMMIT, and ROLLBACK**, and screenshots show proof of rollback behavior.
The project also includes a **complete OpenAPI 3.0 specification**.

Important challenges were solved during development:

- **Concurrent reservations** solved using UPDLOCK
- **Inventory atomicity** solved using SERIALIZABLE
- **Role confusion** solved by separating admin and employee authentication
- **Rollback logging** added for debugging and proof

Key lessons learned:

- Transactions need **dedicated connections**
- UPDLOCK prevents race conditions
- Raw SQL gives more control for complex transactions
- Security should use **multiple layers** like validation, parameterized queries, and database constraints.